🔗 Linked List — Master Revision Sheet (Step 6, P01–P30)

> **One-stop revision.** Each entry = 📖 **Problem** (what's asked + tiny example) + 🧾 **Recall** (the trigger line) + 🔧 **Algorithm** + ⚠️ **Trap / don't-miss** + ⏱️ **Time / Space** + 🧠 **Pattern**. Go top-to-bottom the night before an interview; every linked list problem in this step is here.

⚡ 60-Second Index

#	Problem	LC	Pattern	Time / Space
P01	Introduction to Linked List	—	Node + traversal	O(N) / O(1)
P02	Insert at Head	—	O(1) head insert	O(1) / O(1)
P03	Delete Last Node	—	Stop one early	O(N) / O(1)
P04	Length of Linked List	—	Counter traversal	O(N) / O(1)
P05	Search in Linked List	—	Linear scan	O(N) / O(1)
P06	Introduction to DLL	—	Node + back pointer	O(N) / O(N)
P07	Insert at End of DLL	—	Walk to tail, wire both	O(N) / O(1)
P08	Delete Tail of DLL	—	Use `prev` to detach	O(N) / O(1)
P09	Reverse a DLL	—	Swap next/back per node	O(N) / O(1)
P10	Find Middle of LL	876	Slow & fast	O(N) / O(1)
P11	Reverse a Linked List	206	3-pointer / recursion	O(N) / O(1)
P12	Detect Cycle	141	Floyd's tortoise-hare	O(N) / O(1)
P13	Starting Point of Loop	142	Floyd's two-phase	O(N) / O(1)
P14	Length of Loop	—	Floyd + count one lap	O(N) / O(1)
P15	Check Palindrome	234	Middle + reverse half	O(N) / O(1)
P16	Segregate Even/Odd (by value)	328*	Two dummy chains	O(N) / O(1)
P17	Remove Nth Node from End	19	Two ptr N-gap + dummy	O(N) / O(1)
P18	Delete Middle Node	2095	Slow/fast head-start	O(N) / O(1)
P19	Sort LL	148	Merge sort	O(N log N) / O(log N)
P20	Sort LL of 0s/1s/2s	—	Three dummy chains	O(N) / O(1)
P21	Intersection of Two LL	160	Two-pointer redirect	O(N+M) / O(1)
P22	Add One to LL	—	Reverse/recurse from tail	O(N) / O(1)
P23	Add Two Numbers	2	Dummy + carry	O(max N,M) / O(N)
P24	Delete All Occurrences (DLL)	—	Splice each match	O(N) / O(1)
P25	Find Pairs with Sum (DLL)	—	Two ptr from ends	O(N) / O(1)
P26	Remove Duplicates Sorted DLL	—	Skip adjacent equals	O(N) / O(1)
P27	Reverse Nodes in K-Group	25	Group reverse + reconnect	O(N) / O(1)
P28	Rotate List	61	Ring + cut at len−k	O(N) / O(1)
P29	Flattening a LL	—	Merge-k via child	O(N·M) / O(N)
P30	Copy List w/ Random Pointer	138	Interleave + detach	O(N) / O(1)

* LC #328 is by index; P16 here segregates by value

🧩 Pattern Toolbox (the shapes that repeat)

- **Head-fixed + moving-temp traversal** (P01, P04, P05): never move `head`; walk a `temp` via `while(temp) temp=temp->next`. Backbone of length/search/print.
- **Dummy node** (P17, P19, P20, P23, P30): a sentinel before the head removes all "is this the first node?" special cases; return `dummy->next`.
- **Slow & fast pointers** (P10, P12, P13, P14, P15, P18): fast moves 2× slow → finds middle, detects cycles, locates nth-from-end. Loop guard `fast && fast->next`.
- **Floyd's cycle detection** (P12, P13, P14): detect via collision; loop-start via reset-slow-to-head; loop-length via one lap from the meeting node.
- **3-pointer reversal** (P09 DLL, P11, P22, P27): save `next` before rewriting it; `prev` ends as the new head. DLL variant = swap `next` ↔ `back`.
- **Two dummy chains / partition** (P16, P20): bucket nodes by a predicate then stitch; null-terminate the last chain to avoid cycles.
- **Two-pointer from both ends** (P25): sorted DLL → left/right converge; `back` enables O(1) backward moves.
- **Two-pointer redirect** (P21): on hitting NULL jump to the other head; both walk `len1+len2` → meet at junction.
- **Merge two sorted lists** (P19, P29): dummy + pick-smaller; foundation of merge sort and flatten.
- **Interleave clones** (P30): weave copy after each original so `orig->random->next` = copy's random → O(1)-space deep copy.

P01 — Introduction to Linked List

- 📖 **Problem:** Build a node (data + `next`), access address vs value. `new Node(2)` → `y` holds address, `y->data`=2.
- 🧾 **Recall:** "Node = data + next. `new` gives heap address; `y->data` = `(*y).data`."
- 🔧 **Algo:** Fix `head`, walk `mover`: `mover->next=temp; mover=temp;`. Traverse `while(temp) temp=temp->next`.
- ⚠️ **Trap:** `cout<<y` prints address, not data. Never move `head` while traversing.
- ⏱️ **O(N) build / O(N).**
- 🧠 **Pattern:** Node + traversal skeleton.

P02 — Insert at Head

- 📖 **Problem:** Prepend a value. `2->3`, insert 1 → `1->2->3`.
- 🧾 **Recall:** "New node's next = old head, return new node."
- 🔧 **Algo:** `newNode = new Node(val, head); return newNode;`. Caller reassigns `head`.
- ⚠️ **Trap:** Must **return + reassign** head, else the insert is lost. Works on empty list.
- ⏱️ **O(1) / O(1).**
- 🧠 **Pattern:** O(1) head insert.

P03 — Delete Last Node

- 📖 **Problem:** Remove the tail. `1->2->3` → `1->2`.
- 🧾 **Recall:** "Walk to second-last (`curr->next->next`), `delete curr->next; curr->next=NULL`."
- 🔧 **Algo:** Guard empty/single → return NULL. Else stop one early, delete tail, null the link.
- ⚠️ **Trap:** Single-node guard prevents `curr->next->next` null deref. `delete` before losing the pointer.
- ⏱️ **O(N) / O(1).**
- 🧠 **Pattern:** Stop-one-early traversal.

P04 — Length of Linked List

- 📖 **Problem:** Count nodes. `10->20->30` → 3.
- 🧾 **Recall:** "Walk temp to NULL, count++ each step."
- 🔧 **Algo:** `while(temp){ count++; temp=temp->next; }`.
- ⚠️ **Trap:** No stored size → O(N). Empty list → 0 automatically. Loop on `temp`, not `temp->next`.
- ⏱️ **O(N) / O(1).**
- 🧠 **Pattern:** Counter traversal.

P05 — Search in Linked List

- 📖 **Problem:** Does `key` exist? `10->20->30`, key 20 → true.
- 🧾 **Recall:** "Walk, `if(data==key) return true`, else advance; NULL → false."
- 🔧 **Algo:** Linear scan with early exit on match.
- ⚠️ **Trap:** Return `false` **after** the loop, not on first mismatch. Compare `->data`, not the pointer.
- ⏱️ **O(N) / O(1).**
- 🧠 **Pattern:** Linear scan.

P06 — Introduction to DLL

- 📖 **Problem:** DLL node = data + `next` + `back`; build from array. Two-way links.
- 🧾 **Recall:** "New node's back=prev, then prev->next=new; invariant `a->next==b ⇔ b->back==a`."
- 🔧 **Algo:** `temp=new Node(arr[i],nullptr,prev); prev->next=temp; prev=temp;`.
- ⚠️ **Trap:** Wire **both** directions. Head's `back`=NULL, tail's `next`=NULL.
- ⏱️ **O(N) / O(N).**
- 🧠 **Pattern:** Node + back pointer.

P07 — Insert at End of DLL

- 📖 **Problem:** Append to a DLL tail. `12⇄5`, insert 10 → `12⇄5⇄10`.
- 🧾 **Recall:** "Walk `while(tail->next)`, then `tail->next=new; new->back=tail`."
- 🔧 **Algo:** Guard empty → return newNode. Else reach tail, wire both links.
- ⚠️ **Trap: Insert** stops *on* the tail (`tail->next`); **delete-tail** stops one before (`->next->next`).
- ⏱️ **O(N) / O(1).**
- 🧠 **Pattern:** Walk-to-tail + wire both.

P08 — Delete Tail of DLL

- 📖 **Problem:** Remove DLL tail. `1⇄2⇄3` → `1⇄2`. (Delete-head variant = O(1).)
- 🧾 **Recall:** "Walk to last, `temp->prev->next=NULL; delete temp`."
- 🔧 **Algo:** Guard empty/single. DLL's `prev` avoids tracking second-last.
- ⚠️ **Trap:** Null the new tail's `next`; delete-head must set `newHead->prev=NULL`.
- ⏱️ **O(N) tail / O(1) head.**
- 🧠 **Pattern:** Detach via `prev`.

P09 — Reverse a DLL

- 📖 **Problem:** Reverse in place. `10⇄20⇄30⇄40` → `40⇄30⇄20⇄10`.
- 🧾 **Recall:** "For each node: save next, swap next↔back, advance via saved temp."
- 🔧 **Algo:** `temp=curr->next; curr->next=curr->back; curr->back=temp; head=curr; curr=temp;`.
- ⚠️ **Trap: Save `curr->next` before** swapping. New head = last node processed (old tail).
- ⏱️ **O(N) / O(1).**
- 🧠 **Pattern:** Swap next/back per node.

P10 — Find Middle of LL

- 📖 **Problem:** Middle node; 2nd middle if even. `1→2→3→4→5` → 3.
- 🧾 **Recall:** "Slow +1, fast +2, `while(fast && fast->next)`; slow lands middle."
- 🔧 **Algo:** Both start at head → 2nd middle on even length.
- ⚠️ **Trap:** Check `fast` before `fast->next`. Advancing fast by 1 → slow at ¼, not middle.
- ⏱️ **O(N) / O(1).**
- 🧠 **Pattern:** Slow & fast.

P11 — Reverse a Linked List

- 📖 **Problem:** Reverse singly LL. `1→2→3` → `3→2→1`.
- 🧾 **Recall:** "Iter: `front=temp->next; temp->next=prev; prev=temp; temp=front;` return prev."
- 🔧 **Algo:** Iterative 3-pointer, or recursive (reverse rest, `head->next->next=head; head->next=NULL`).
- ⚠️ **Trap:** Return `prev`, not `head`. Recursive must null the tail or you get a cycle.
- ⏱️ **O(N) / O(1) iter, O(N) recursive stack.**
- 🧠 **Pattern:** 3-pointer / recursion.

P12 — Detect Cycle

- 📖 **Problem:** Is there a loop? true/false.
- 🧾 **Recall:** "Slow +1, fast +2. Ever meet → cycle; fast hits NULL → none."
- 🔧 **Algo:** Floyd's; gap shrinks by 1 each step inside a loop → guaranteed collision.
- ⚠️ **Trap:** Test `slow==fast` **after** moving. Compare node addresses, not values.
- ⏱️ **O(N) / O(1).**
- 🧠 **Pattern:** Floyd's tortoise-hare.

P13 — Starting Point of Loop

- 📖 **Problem:** Return the loop's entry node (or NULL).
- 🧾 **Recall:** "Detect meeting, reset slow=head, move both +1 → meet at loop start."
- 🔧 **Algo:** `L = kC − d` ⇒ head-distance equals meeting-to-start distance.
- ⚠️ **Trap:** Phase 2 moves both by **1** (not 2); reset only **slow** to head.
- ⏱️ **O(N) / O(1).**
- 🧠 **Pattern:** Floyd's two-phase.

P14 — Length of Loop

- 📖 **Problem:** Count nodes in the loop.
- 🧾 **Recall:** "Floyd meet inside loop, walk from meeting node back to itself counting."
- 🔧 **Algo:** `len=1; while(temp->next!=meet){temp=temp->next; len++;}`.
- ⚠️ **Trap:** Start `len=1` (counts start node). Count from meeting point, not head.
- ⏱️ **O(N) / O(1).**
- 🧠 **Pattern:** Floyd + one lap.

P15 — Check Palindrome

- 📖 **Problem:** Reads same both ways? `1→5→2→5→1` → true.
- 🧾 **Recall:** "Find middle, reverse 2nd half, compare, reverse back."
- 🔧 **Algo:** Middle-finder `while(fast->next && fast->next->next)` → slow at end of first half.
- ⚠️ **Trap:** Compare while `second != NULL` (odd middle auto-skipped). Restore the list.
- ⏱️ **O(N) / O(1).**
- 🧠 **Pattern:** Middle + reverse half.

P16 — Segregate Even/Odd (by value)

- 📖 **Problem:** Even-valued nodes first, then odd. `1→2→3→4` → `2→4→1→3`.
- 🧾 **Recall:** "Two dummy chains by `val&1`, then `evenTail->next=oddHead->next`."
- 🔧 **Algo:** Detach each node (`temp->next=nullptr`), append to even/odd tail, stitch.
- ⚠️ **Trap:** Value-parity ≠ LC #328 index-parity. Null on detach or you get cycles.
- ⏱️ **O(N) / O(1).**
- 🧠 **Pattern:** Two dummy chains.

P17 — Remove Nth Node from End

- 📖 **Problem:** Delete Nth-from-end. `1→2→3→4→5`, N=3 → `1→2→4→5`.
- 🧾 **Recall:** "Dummy; fast +N+1; move both till fast NULL; `slow->next=slow->next->next`."
- 🔧 **Algo:** N-gap between fast/slow; slow lands before target.
- ⚠️ **Trap:** N+1 (not N) steps; dummy kills the delete-head case; return `dummy->next`.
- ⏱️ **O(N) one pass / O(1).**
- 🧠 **Pattern:** Two ptr N-gap + dummy.

P18 — Delete Middle Node

- 📖 **Problem:** Delete the middle. `1→2→3→4→5` → `1→2→4→5`.
- 🧾 **Recall:** "fast=head->next->next → slow stops before middle → bypass + delete."
- 🔧 **Algo:** Head-start on fast shifts slow back one; loop `fast && fast->next`.
- ⚠️ **Trap:** Single-node guard also protects `head->next->next`. Delete the unlinked node.
- ⏱️ **O(N) / O(1).**
- 🧠 **Pattern:** Slow/fast head-start.

P19 — Sort LL

- 📖 **Problem:** Sort ascending in O(N log N). `3→2→5→4→1` → `1→2→3→4→5`.
- 🧾 **Recall:** "Merge sort: findMiddle, split, recurse both, merge with dummy."
- 🔧 **Algo:** `findMiddle` uses `fast=head->next`; split `middle->next=nullptr`.
- ⚠️ **Trap:** `fast=head` → size-2 never splits → infinite recursion. Split before recursing.
- ⏱️ **O(N log N) / O(log N) stack.**
- 🧠 **Pattern:** Merge sort.

P20 — Sort LL of 0s/1s/2s

- 📖 **Problem:** Sort {0,1,2} by relinking. `1→2→0→1→2→0` → `0→0→1→1→2→2`.
- 🧾 **Recall:** "Three dummy chains, stitch zero→one→two (skip empty ones), `twoTail->next=null`."
- 🔧 **Algo:** Bucket by value; conditional stitch when no 1s exist.
- ⚠️ **Trap: Must** null the twos tail (old links → cycle). `head=zeroDummy->next`.
- ⏱️ **O(N) / O(1).**
- 🧠 **Pattern:** Three dummy chains.

P21 — Intersection of Two LL

- 📖 **Problem:** First common node (Y-merge) or NULL.
- 🧾 **Recall:** "Two ptrs; on NULL jump to other head; meet at junction."
- 🔧 **Algo:** Both walk `len1+len2` → align. Or length-diff, or hash set.
- ⚠️ **Trap:** Compare **node addresses**, not values. Switch at NULL before advancing.
- ⏱️ **O(N+M) / O(1).**
- 🧠 **Pattern:** Two-pointer redirect.

P22 — Add One to LL

- 📖 **Problem:** Number as MSD-first list, add 1. `1→2→9` → `1→3→0`.
- 🧾 **Recall:** "Reverse→add carry→reverse back, OR recurse (base returns 1), prepend if head carries."
- 🔧 **Algo:** Addition flows from tail; carry init = 1.
- ⚠️ **Trap:** Reverse **back** in iterative form. Handle surviving carry (`999→1000`).
- ⏱️ **O(N) / O(1) iter.**
- 🧠 **Pattern:** Reverse/recurse from tail.

P23 — Add Two Numbers

- 📖 **Problem:** LSD-first lists, return sum list. `342 + 465` → `7→0→8`.
- 🧾 **Recall:** "`while(l1||l2||carry)` : sum digits+carry, node= `sum%10`, carry= `sum/10`."
- 🔧 **Algo:** Dummy builds result; digits already reversed → no reversal.
- ⚠️ **Trap:** Keep `|| carry` in the loop (final carry node). Guard unequal lengths.
- ⏱️ **O(max N,M) / O(N).**
- 🧠 **Pattern:** Dummy + carry.

P24 — Delete All Occurrences (DLL)

- 📖 **Problem:** Remove every node == key. key=2: `2⇄3⇄2⇄4` → `3⇄4`.
- 🧾 **Recall:** "On match splice `prev->next=next; next->back=prev`, move head if it's head, delete."
- 🔧 **Algo:** Single pass; save `next` before deleting, resume from it.
- ⚠️ **Trap:** Update `head` on head-deletion; guard null neighbours at the ends.
- ⏱️ **O(N) / O(1).**
- 🧠 **Pattern:** Splice each match.

P25 — Find Pairs with Sum (sorted DLL)

- 📖 **Problem:** All pairs summing to target. sum=7: `1⇄2⇄3⇄4⇄5⇄6` → (1,6)(2,5)(3,4).
- 🧾 **Recall:** "left=head/right=tail; ==sum record+move both; <sum left++; >sum right--."
- 🔧 **Algo:** Two pointers from both ends; `back` enables backward moves.
- ⚠️ **Trap:** Stop `left!=right && right->next!=left` (odd & even crossing). Needs sorted.
- ⏱️ **O(N) / O(1).**
- 🧠 **Pattern:** Two ptr from ends.

P26 — Remove Duplicates from Sorted DLL

- 📖 **Problem:** Dedupe adjacent equals. `1⇄1⇄3⇄3⇄3⇄4` → `1⇄3⇄4`.
- 🧾 **Recall:** "Skip+delete equal-valued next nodes, then `temp->next=nextNode; nextNode->back=temp`."
- 🔧 **Algo:** Sorted → dups contiguous; one sweep clears each value.
- ⚠️ **Trap:** Advance `nextNode` before `delete`; restore both links; null guard at tail.
- ⏱️ **O(N) / O(1).**
- 🧠 **Pattern:** Skip adjacent equals.

P27 — Reverse Nodes in K-Group

- 📖 **Problem:** Reverse every k nodes; leftover < k kept. k=2: `1→2→3→4→5` → `2→1→4→3→5`.
- 🧾 **Recall:** "Find kth (null → attach tail), cut, reverse, `prevLast->next=kthNode`, prevLast=old head."
- 🔧 **Algo:** After reverse: kthNode=new head, temp=new tail. First group sets `head`.
- ⚠️ **Trap:** Don't reverse the incomplete last group. Cut `kthNode->next=nullptr` before reversing.
- ⏱️ **O(N) / O(1).**
- 🧠 **Pattern:** Group reverse + reconnect.

P28 — Rotate List

- 📖 **Problem:** Rotate right by k. `1→2→3→4→5`, k=2 → `4→5→1→2→3`.
- 🧾 **Recall:** "Count length, ring (tail->next=head), k%=len, walk len−k → new tail, cut."
- 🔧 **Algo:** New head at position `length - k`; sever the ring after.
- ⚠️ **Trap:** `k %= length` (full rotations cancel). Must cut the ring or infinite list.
- ⏱️ **O(N) / O(1).**
- 🧠 **Pattern:** Ring + cut at len−k.

P29 — Flattening a LL

- 📖 **Problem:** Child sub-lists sorted; flatten to one sorted `child` chain.
- 🧾 **Recall:** "Recurse right via next, merge each column into result through child pointers."
- 🔧 **Algo:** Merge-k-sorted; merge threads through `child`, nulls `next`.
- ⚠️ **Trap:** Merge via `child` not `next`; null `next` or cycles form.
- ⏱️ **O(N·M) / O(N) stack.**
- 🧠 **Pattern:** Merge-k via child.

P30 — Copy List with Random Pointer

- 📖 **Problem:** Deep-copy list with `random` pointers, independent of original.
- 🧾 **Recall:** "Weave A→A'→B→B', copy random = `orig->random->next`, detach."
- 🔧 **Algo:** Three passes: insert copies, link randoms, separate into two lists.
- ⚠️ **Trap:** `copy->random = temp->random->next` (copy→copy). Guard null random. Restore original's `next`.
- ⏱️ **O(N) / O(1) extra.**
- 🧠 **Pattern:** Interleave + detach.

🎯 The 8 Reusable Templates

1. **Traversal skeleton** — `Node* temp=head; while(temp){ ...; temp=temp->next; }` (P01, P04, P05).
2. **Dummy-node builder** — `dummy; temp=dummy; ...; return dummy->next;` (P17, P19, P20, P23, P30).
3. **Slow/fast** — `while(fast && fast->next){ slow=slow->next; fast=fast->next->next; }` (P10, P12–15, P18).
4. **3-pointer reverse** — `front=temp->next; temp->next=prev; prev=temp; temp=front;` (P11, P22, P27).
5. **Floyd's two-phase** — collision, then reset slow=head, step both by 1 (P12, P13, P14).
6. **Merge two sorted** — dummy + pick-smaller + attach leftover (P19, P29).
7. **Partition into dummy chains** — bucket by predicate, stitch, null-terminate (P16, P20).
8. **DLL splice** — `prev->next=next; next->back=prev;` with null guards (P24, P26).

⏱️ 5-Minute Night-Before Skim

- **Singly basics (P01–P05):** traversal skeleton, O(1) head insert, stop-one-early delete.
- **DLL (P06–P09, P24–P26):** always wire/splice **both** `next` and `back`; guard nulls at ends.
- **Slow/fast (P10, P12–P15, P18):** middle, cycle detect/start/length, palindrome, delete-middle — all the same two pointers.
- **Dummy node (P17, P19, P20, P23, P30):** kills first-node edge cases.
- **Reversal family (P09, P11, P22, P27):** save next before rewriting; DLL swaps next↔back.
- **Hard four (P27–P30):** k-group reverse (prevLast reconnection), rotate (ring + len−k), flatten (merge-k via child), copy-random (interleave + detach, O(1) space).